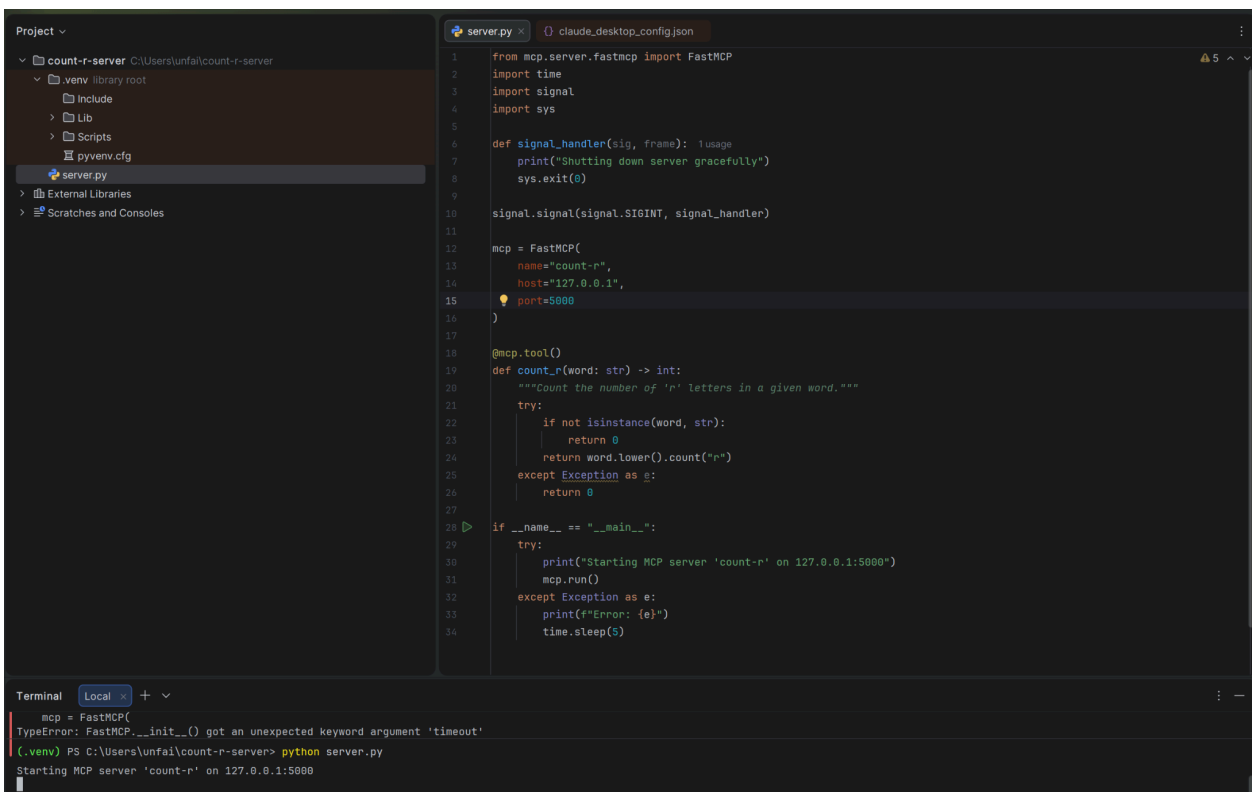
Noe Team 3 - Week 2 Report

Team Progress

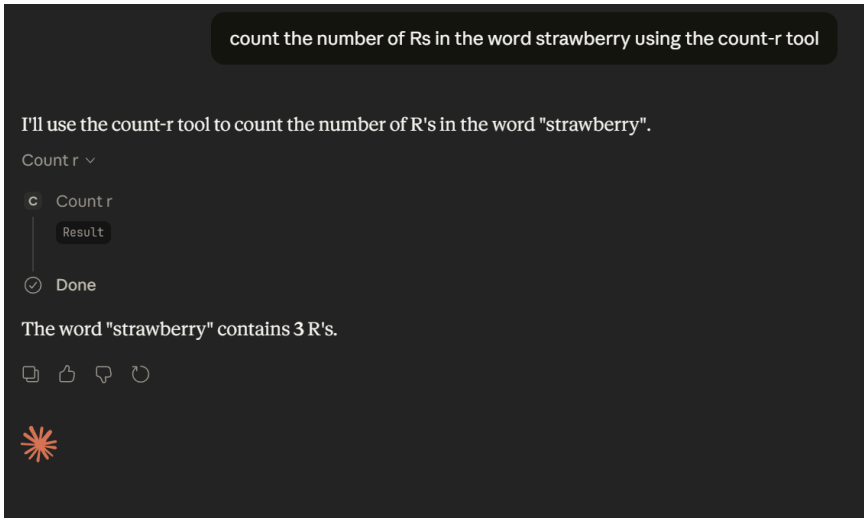
Calvin Beechner

I focused on finding a model for our group to use with the NASA core Flight Software. I propose we use Claude Desktop's Sonnet 4.5. I learned how to create an MCP server by following the tutorial from this video, https://youtu.be/wa_A0qY0anA?si=OoEovEZpg2Yo2_i-. What I found was that you can easily set up an MCP server using python and then use tools we define in that server via a Claude query.

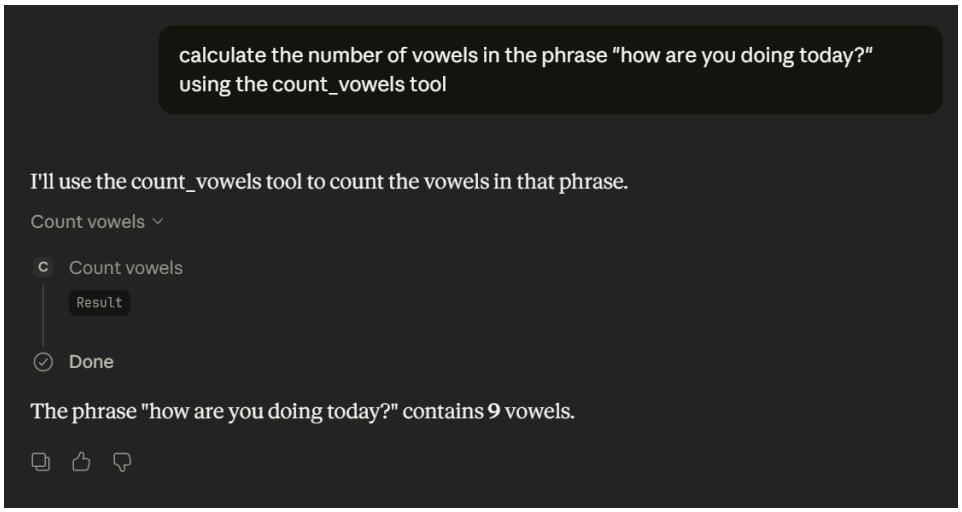
Here's the code I wrote in python. This was used to set up a server.



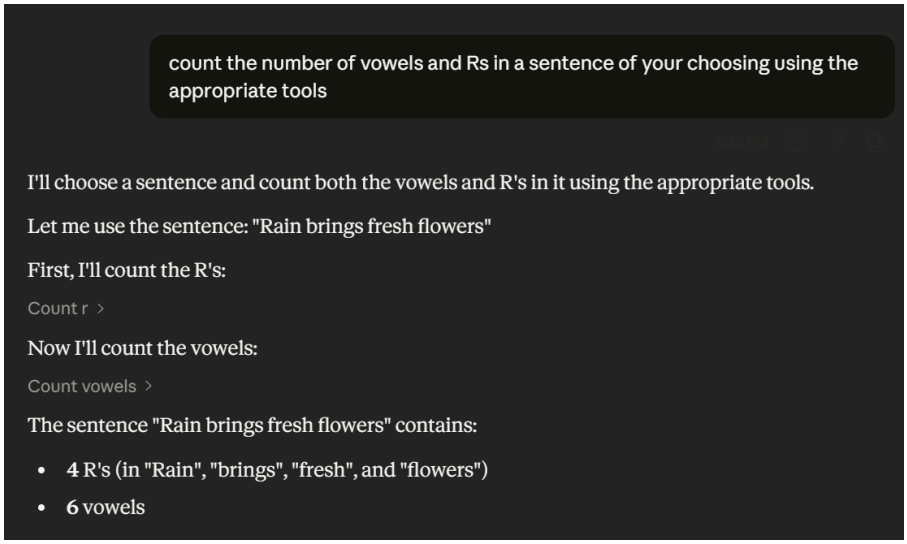
Here's the prompt I sent to Claude which used my MCP Server and my defined tool



From there, I created my own tool, count_vowels, which counted the number of instances of letters from the set [a,e,i,o,u].



At this point I knew the model was capable of executing my code with MCP but I wanted to know if it could choose to use a tool at its own discretion. I gave it a task that involved both tools, but I did not specify a word nor did I specify what tools to use.



From this we see the model was able to create its own sentence and then call both tools.

Next, I asked the model to count the number of consonants using the appropriate tool. At first the model claimed it was incapable of fulfilling that request due to there being no appropriate tool. However, after I gave it an assumption that all letters are either vowels or consonants it completed my request and remembered the assumption for later requests.

Count the number of constants in a sentance of your choosing using the appropriate tools

I'll choose a sentence and try to count the consonants using the appropriate tools.

Let me use the sentence: "Hello world"

Used 2 tools >

Unfortunately, there doesn't appear to be a tool available for counting consonants. The tools I have access to are:

- `count_vowels` - for counting vowels
- `count_r` - for counting the letter R specifically

I don't have a general consonant-counting tool available. If you'd like, I could count the vowels in a sentence instead, or manually count the consonants for you without using a tool.

Given the assumption that all letters are either vowels or constants, count the number of constants in a sentence of your choosing using the appropriate tools

Ah, clever! If all letters are either vowels or consonants, I can count the vowels and then calculate the consonants. Let me choose a sentence and use the available tools.

Let me use the sentence: "Hello world"

First, I'll count the vowels:

Count vowels >

Now I need to count the total number of letters in "Hello world":

- Removing spaces: "Helloworld" = 10 letters total

- Removing spaces: "Helloworld" = 10 letters total

Since there are 3 vowels and 10 total letters:

- **Consonants = 10 - 3 = 7**

The sentence "Hello world" contains **7 consonants** (calculated by subtracting the 3 vowels from the 10 total letters).

Count the number of consonants in a sentence of your choosing using the appropriate tools

I'll choose a sentence and count the consonants using the available tools.

Let me use the sentence: "Programming is fun"

First, I'll count the vowels:

Count vowels >

Now I'll count the total letters in "Programming is fun":

- Removing spaces: "Programmingisfun" = 16 letters total

Since there are 5 vowels and 16 total letters:

- **Consonants = 16 - 5 = 11**

The sentence "Programming is fun" contains **11 consonants** (calculated by subtracting the 5 vowels from the 16 total letters).

After this I've learned that the models are capable of determining the appropriate time to use a tool, and they can learn to use a tool for an undefined task.

In summary, I decided we will use Claude's Sonnet 4.5 and I practiced creating an MCP server using FastMCP.

Jose Varela

I have been trying to get a further understanding of how the NASA core Flight Software works. This link provides a good brief overview: https://www.youtube.com/watch?v=gSsV6fDEcAc , https://www.youtube.com/watch?v=-_gUwzQahH4.

I also went into depth regarding zero trust and how it applies to the MCP in satellites. Zero Trust treats every request as hostile until it proves otherwise and space systems make that proof slow, expensive, and sometimes impossible.

After meeting with our advisor, we decided to divide and conquer so Calvin will try and analyze the threat model.

But, overall as of now I have watched alot of videos and read published articles over the NASA core flight software. In the next day or two, I want to dive deep into the github that is opensource from the nasa core flight software and see the ideas that are in it. It might be a good idea to meet with my team to present our findings and see where we can go from there.

Downloadable Executive: https://software.nasa.gov/software/GSC-18128-1

Publications CFS: https://etd.gsfc.nasa.gov/capabilities/core-flight-system/publications

Jesus Castro - Garcia

This I have been researching how to download cfs onto our computers so after looking at multiple videos and sources, I found out that we will need a virtual machine which I used VMmare Workstation. I also downloaded Ubuntu as that is the virtual machine that they use. After having it downloaded and setting things up. I cloned the nasa repo:

```
jesus1006@jesus1006-VMware-Virtual-Platform:~$ git clone http://www.github.com/n
asa/cFS.git
Cloning into 'cFS'...
warning: redirecting to https://github.com/nasa/cFS.git/
remote: Enumerating objects: 2310, done.
remote: Counting objects: 100% (74/74), done.
remote: Compressing objects: 100% (40/40), done.
remote: Total 2310 (delta 46), reused 39 (delta 34), pack-reused 2236 (from 2)
Receiving objects: 100% (2310/2310), 6.13 MiB | 7.16 MiB/s, done.
Resolving deltas: 100% (1195/1195), done.
jesus1006@jesus1006-VMware-Virtual-Platform:~$ dir
```

From here I was able to access the files that are in the github.

I ran this command ‘make SIMULATION=native prep’ to prepare the build.

```
jesus1006@jesus1006-VMware-Virtual-Platform:~$ cd cFS
jesus1006@jesus1006-VMware-Virtual-Platform:~/cFS$ make SIMULATION=native prep
mkdir -p "build"
(cd "build" && cmake -DCMAKE_INSTALL_PREFIX=/exe -DCMAKE_BUILD_TYPE=debug "/home
/jesus1006/cFS/cfe")
-- OMIT_DEPRECATED=false: Deprecated elements included in build
-- Search path for modules: apps;libs;psp/fsw/modules;cfe/modules
-- Module 'core_api' found at /home/jesus1006/cFS/cfe/modules/core_api
-- Module 'core_private' found at /home/jesus1006/cFS/cfe/modules/core_private
-- Module 'ci_lab' found at /home/jesus1006/cFS/apps/ci_lab
-- Module 'to_lab' found at /home/jesus1006/cFS/apps/to_lab
-- Module 'sch_lab' found at /home/jesus1006/cFS/apps/sch_lab
-- Module 'cfe_assert' found at /home/jesus1006/cFS/cfe/modules/cfe_assert
-- Module 'cfe_testcase' found at /home/jesus1006/cFS/cfe/modules/cfe_testcase
-- Module 'sample_app' found at /home/jesus1006/cFS/apps/sample_app
-- Module 'sample_lib' found at /home/jesus1006/cFS/libs/sample_lib
-- Module 'es' found at /home/jesus1006/cFS/cfe/modules/es
-- Module 'evs' found at /home/jesus1006/cFS/cfe/modules/evs
-- Module 'fs' found at /home/jesus1006/cFS/cfe/modules/fs
-- Module 'sb' found at /home/jesus1006/cFS/cfe/modules/sb
-- Module 'tbl' found at /home/jesus1006/cFS/cfe/modules/tbl
-- Module 'time' found at /home/jesus1006/cFS/cfe/modules/time
-- Module 'osal' found at /home/jesus1006/cFS/osal
```

Then I did ‘make’ to compile the source code.

```
jesus1006@jesus1006-VMware-Virtual-Platform:~/cFS$ make
make --no-print-directory -C "build" mission-all
[ 22%] Built target elf2cfetbl
[ 22%] Built target cfe-build-env
[ 22%] Built target cfe-module-version
[ 22%] Built target mission-version
[ 33%] Built target cfeconfig_platformdata_native_default_cpu1
[ 77%] Built target cfeconfig_platformdata_tool
[100%] Built target cfgtool-execute-native_default_cpu1
[100%] Built target mission-prebuild
[ 33%] Built target tblobj_cpu1_sample_app.tblda5677466155556303414379 26dc732477
31b62a
[ 66%] Generating /home/jesus1006/cFS/build/tables/cpu1_sample_app.tblda56774661
5555630341437926dc73247731b62a.sample_app_tbl.d
[ 66%] Built target generate_table_cpu1_sample_app_sample_app_tbl
[100%] Built target tblobj_cpu1_to_lab.tbl981354792926b14a093231ef5fd1792d942ba0
63
[100%] Generating /home/jesus1006/cFS/build/tables/cpu1_to_lab.tbl981354792926b1
4a093231ef5fd1792d942ba063.to_lab_sub.d
[100%] Built target generate_table_cpu1_to_lab_to_lab_sub
```

Next I ran ‘make install’.

```
jesus1006@jesus1006-VMware-Virtual-Platform:~/cFS$ make install
make --no-print-directory -C "build" DESTDIR="/home/jesus1006/cFS/build" mission
-install
[ 22%] Built target elf2cfetbl
[ 22%] Built target cfe-build-env
[ 22%] Built target cfe-module-version
[ 22%] Built target mission-version
[ 33%] Built target cfeconfig_platformdata_native_default_cpu1
[ 77%] Built target cfeconfig_platformdata_tool
[100%] Built target cfgtool-execute-native_default_cpu1
[100%] Built target mission-prebuild
[ 33%] Built target tblobj_cpu1_sample_app.tblda567746615555630341437926dc732477
31b62a
[ 66%] Built target generate_table_cpu1_sample_app_sample_app_tbl
[100%] Built target tblobj_cpu1_to_lab.tbl981354792926b14a093231ef5fd1792d942ba0
63
[100%] Built target generate_table_cpu1_to_lab_to_lab_sub
[100%] Built target tblobj_cpu1_sch_lab.tblf24302c415d68c922ed1ac8020f6ac82b5bc5
f2f
[100%] Built target generate_table_cpu1_sch_lab_sch_lab_table
[100%] Built target tblobj_cpu1_cfeTestAppTable.tble4548dd84ad8fd883dc34e11b0264
edac0a8f362
[100%] Built target generate_table_cpu1_cfeTestAppTable_cfe_test_tbl
```

From there I navigated to the build directory with ‘build/exe/cpu1/’ and ‘executed ./core-cpu1’ to run the simulation.

```
jesus1006@jesus1006-VMware-Virtual-Platform:~/cFS$ cd build/exe/cpu1
jesus1006@jesus1006-VMware-Virtual-Platform:~/cFS/build/exe/cpu1$ ./core-cpu1
OS_BSP_Initialize():Maximum user msg queue depth = 10
CFE_PSP: Default Reset SubType = 1
CFE_PSP: Default CPU ID = 1
CFE_PSP: Default Spacecraft ID = 66
CFE_PSP: Default CPU Name: cpu1
OS_Posix_GetSchedulerParams():202:Policy 1: available, min-max: 1-99
OS_Posix_GetSchedulerParams():202:Policy 2: available, min-max: 1-99
OS_Posix_TaskAPI_Impl_Init():388:Selected policy 2 for RT tasks, root task = 99
OS_Posix_TaskAPI_Impl_Init():405:Could not setschedparam in main thread: Operati
on not permitted (1)
CFE_PSP: initializing module 'soft_timebase' with ID 0110ff00
CFE_PSP: Instantiated software timebase 'cFS-Master' running at 10000 usec
CFE_PSP: initializing module 'timebase_posix_clock' with ID 0110ff01
CFE_PSP: Using POSIX monotonic clock as CFE timebase
CFE_PSP: initializing module 'eeprom_mmap_file' with ID 0110ff02
CFE_PSP: Using MMAP simulated EEPROM implementation
CFE_PSP: initializing module 'ram_notimpl' with ID 0110ff03
CFE_PSP: EEPROM Range (2) created: Start Address = 7DB133423000, Size = 00080000
 Status = 0
CFE_PSP: Physical RAM access not implemented
```

It is a start with being able to run the simulation but will still need to work out how to be able to implement with MCP and do further testing

Sebastian Alturck-Carlos

I have been trying to figure out how the structure for our project would be so at a very high level. I have this:

AI Agent (Host)

↓

MCP Client

↓

YOUR MCP Server

↓

Simulator Adapter Layer

↓

Flight Simulator

And a very dumbed down version of code we'd need to implement is as follows

```
from mcp.server import Server
import simulator_adapter

server = Server("satellite-control")

@server.tool()
def execute_maneuver(delta_v: float, duration: float):
    return simulator_adapter.run_burn(delta_v, duration)

@server.tool()
def get_telemetry():
    return simulator_adapter.read_state()

server.run()
```

We are looking to get an MCP server that we can connect to the Simulator to then test it's security. I also found a solid MCP that we can use for this project as a suggestion.

https://github.com/microsoft/mcp-for-beginners

Noe Rosales

For my task this week, I have dedicated my time to researching the current risk and challenges MCP poses. Throughout my research I have encountered numerous issues that MCP currently has, but a recurring issue that I have noticed is the confused deputy problem. I learned that this problem involves a privileged program being misused by a less-privileged user. This issue is a form of privilege escalation that often occurs in cross-service scenarios.

That alone is not the only issue, as https://adversa.ai/mcp-security-top-25-mcp-vulnerabilities/ list 25 current issues that plagues MCP. Below is part of the chart listed on the website.

Rank	Category	Specificity	Component	Name	Alternative Names	Impact Score	Exploitability	Links
1	Input/Instruction Boundary Distinction Failure	AI	Both	**Prompt Injection**	Indirect Prompt Injection, Instruction Hijacking, Context Hijacking	Critical (10/10)	Trivial	Link
2	Input Validation/Sanitization Failures	AppSec	MCP Server	**Command Injection**	OS Command Injection, Shell Injection, System Call Injection	Critical (10/10)	Easy	Link
3	Input/Instruction Boundary Distinction Failure	Unique	MCP Server	**Tool Poisoning (TPA)**	Malicious Tool Descriptor, Function Injection, Basic Tool Poisoning, Metadata Poisoning	Critical (9/10)	Easy	Link
4	Input Validation/Sanitization Failures	AppSec	Both	**Remote Code Execution**	RCE, Arbitrary Code Execution	Critical (10/10)	Moderate	Link
5	Missing Authentication/Authorization Framework	Both	Both	**Unauthenticated Access**	Unrestricted URL, Zero-Auth Vulnerability	Critical (9/10)	Trivial	Link
6	Session Management Design Flaw	Both	MCP Server	**Confused Deputy (OAuth Proxy)**	Deputy Confusion Attack, OAuth Token Confusion, OAuth Proxy Attack, Static Client ID Vulnerability, Consent Cookie Bypass	Critical (9/10)	Moderate	Link
7	Missing Integrity/Verification Controls	Unique	MCP Client	**MCP Configuration Poisoning**	MCPoison, Config Manipulation, Config Injection	High (8/10)	Moderate	Link
8	Missing Authentication/Authorization Framework	Both	Both	**Token/Credential Theft**	Credential Leakage, Token Exposure, Exposed Credentials, Secret Exfiltration	High (8/10)	Easy	Link
9	Session Management Design Flaw	Both	MCP Server	**Token Passthrough**	Token Relay Attack, Credential Forwarding,Token Forwarding, Audience Validation Failure, Improper Token Delegation	High (8/10)	Easy	Link
10	Input Validation/Sanitization Failures	AppSec	MCP Server	**Path Traversal**	Directory Traversal, Dot-Dot-Slash Attack, EscapeRoute Attack, Directory Containment Bypass, Symlink Bypass	High (8/10)	Moderate	Link
11	Input/Instruction Boundary Distinction Failure	Unique	MCP Server	**Full Schema Poisoning (FSP)**	Schema-wide Injection, Extended Tool Poisoning, Schema Injection, Metadata Manipulation	High (8/10)	Moderate	Link
12	Missing Integrity/Verification Controls	Unique	MCP Client	**Tool Name Spoofing**	Homoglyph Attack, Typosquatting, Tool Impersonation, Service Masquerading, Typosquatting Tool Names	High (7/10)	Moderate	Link
13	Network Binding/Isolation Failures	Both	Both	**Localhost Bypass (NeighborJack)**	Network Exposure, 0.0.0.0 Vulnerability, DNS Rebinding, 0.0.0.0 Day Attack, Network Binding Misconfiguration, 0.0.0.0 Exposure	High (8/10)	Moderate	Link
14	Missing Integrity/Verification Controls	Unique	MCP Server	**Rug Pull Attack**	Update Attack, Dynamic Tool Mutation, Fake Updates, Supply Chain Subversion, Tool Update Hijack	High (7/10)	Easy	Link

There are 4 sections in this chart which are Critical, High, Medium, and Low. Currently, the most concerning issues are the ones in Critical and High.

I have also read additional research papers such as https://arxiv.org/pdf/2503.23278? which identifies risks they found while using OpenAi's LLM.

Teams Plan To Catch Up

In week 3, we plan to meet with Dr. Chen and Dr. Zieglar. We all need to debrief each other on our current research and start to put our ideas together and see what we can do with starting an implementation or continuing our research. We will also start meeting more as a team to better strengthen our team communication.

Teams Plan For Next Week

Sebastian Alturck-Carlos

I want to further investigate the MCP layer of our project and see the best way to connect the Simulator and MCP. This is obviously not a 1 man job so collaboration would be needed. I want to also test the SIM its self to get a good idea of what functions we want automated.

Calvin Beechner

Next week, I plan to try and connect the NASA Flight Core software to our MCP project. I will meet with the group for this task as we need to decide on an MCP implementation. I noticed Sebastian used Microsoft's MCP for beginners while I used FastMCP. We will meet and compare the two to decide which implementation we will use going forward.

Jesus Castro - Garcia

I will plan to talk with the rest of the team on how to implement the NASA flight core with MCP and talk about how we could implement the AI model once we reach that part of the project. I will also do more playing around with the CFS as I run it but not exactly sure on what my output is supposed to be. I have a start on the program so that makes things easier but I definitely want to see how it works now that I have the code in my computer and will make it easier.

Noe Rosales

Since we have determined to test with Cloude, I plan on looking into any research that has been done with that current LLM. Additionally, I want to further deep dive into https://arxiv.org/pdf/2503.23278?, and see if I can replicate any of their issues with Claude. I also want to research current risk when it comes to satellite communication, to see if any of those challenges pose a risk for our current implementation plan. Finally, I want to research current solutions that have been proposed for fixing any of the issues in the Critical and High sections of the chart shown above.

Jose Varela

I want to see if me and my team can get something started (“Hello World”) and start to do some bare bones with our implementation. My research is very broad with CFS and my goal is to further narrow it. I have mixed feelings with the CFS implementation, so understanding it more or looking at other paths with it might be a good idea. I also plan to start a presentation template where we can slowly start putting our ideas together.